Fiche d'exercices — Programmation orientée objet en Python

Classes, attributs, méthodes · Encapsulation et héritage · BTS CIEL

Nom : .....

Classe : .....

> **Mémo** : **Classe** : modèle. **Objet** (instance) : réalisation concrète de ce modèle. **Attribut** : donnée portée par l'objet. **Méthode** : fonction définie dans la classe. `self` désigne l'objet sur lequel la méthode est appelée ; `__init__` est le **constructeur**. `return` *retourne* une valeur, `print` l'*affiche*.

Partie A — Classes, attributs et méthodes

Exercice 1 — La classe `Voiture`

```python
class Voiture:
    def __init__(self, marque, couleur):
        self.marque = marque
        self.couleur = couleur

    def afficher_details(self):
        return f"Voiture {self.marque} de couleur {self.couleur}"
```

Questions

1. Que fait la méthode `__init__` ?
2. Combien d'attributs possède la classe `Voiture` ?
3. Que retourne la méthode `afficher_details` ?
4. Créez une instance de `Voiture` de marque `"Toyota"` et de couleur `"Bleu"`, appelez `afficher_details` et affichez le résultat.

Réponses

Exercice 2 — Compléter la classe `Chien`

```python
class Chien:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age

    # Méthode aboyer qui retourne "Wouf !"
```

Questions

1. Quels sont les attributs de la classe `Chien` ?
2. Comment appelle-t-on la méthode `__init__` dans le contexte de la POO ?
3. Ajoutez une méthode `aboyer` qui retourne `"Wouf !"`.
4. Créez un objet `Chien` nommé `"Rex"`, âgé de 3 ans, et faites-le aboyer.

Réponses

Exercice 3 — La classe `Livre`

```python
class Livre:
    def __init__(self, titre, auteur):
        self.titre = titre
        self.auteur = auteur

    def description(self):
        return f"{self.titre} par {self.auteur}"
```

Questions

1. Que fait la méthode `description` ?
2. Combien de paramètres la méthode `__init__` attend-elle, en incluant `self` ?
3. Créez un objet `Livre` de titre `"Le Petit Prince"` et d'auteur `"Antoine de Saint-Exupéry"`, puis affichez sa description.

Réponses

Exercice 4 — La classe `Rectangle`

```python
class Rectangle:
    def __init__(self, longueur, largeur):
        self.longueur = longueur
        self.largeur = largeur

    # Méthode calculer_aire qui retourne l'aire
```

Questions

1. Quels sont les attributs de la classe `Rectangle` ?
2. Quelle formule mathématique devez-vous utiliser pour calculer l'aire ?
3. Ajoutez une méthode `calculer_aire` qui retourne l'aire du rectangle.
4. Créez un rectangle de longueur 5 et de largeur 3, puis affichez son aire.

Réponses

Exercice 5 — Création complète d'une classe `Etudiant`

On souhaite modéliser un étudiant par son nom et sa note.

Questions

1. Créez une classe `Etudiant` avec un constructeur `__init__` prenant en paramètres `nom` et `note`, et deux attributs d'instance `self.nom` et `self.note`.
2. Ajoutez une méthode `a_reussi` qui retourne `True` si la note est supérieure ou égale à 10, et `False` sinon.
3. Testez votre classe en créant un étudiant avec une note de 8.
4. Ajoutez une méthode `augmenter_note` qui augmente la note de 2 points.
5. Créez un étudiant nommé `"Alice"` avec une note de 15 et affichez si elle a réussi, grâce à la méthode `a_reussi`.

Réponses

Partie B — Encapsulation et héritage

Mémo : **Encapsulation** : `self.__nom` (deux soulignés) déclenche le *name mangling* — Python renomme l'attribut en `_Classe__nom`, donc inaccessible de l'extérieur par son nom d'origine. On y accède par un **accesseur** (`get_nom`) et on le modifie par un **mutateur** (`set_nom`), qui peut valider la valeur. **Héritage** : `class Chat(Animal)` — la classe fille reçoit les attributs et méthodes de la mère, peut en ajouter ou **redéfinir** les siennes ; `super().__init__(...)` appelle le constructeur de la mère.

Exercice 6 — Encapsulation simple avec attribut privé

```
class Personne:
    def __init__(self, nom, age):
        self.__nom = nom
        self.__age = age

    def get_nom(self):
        return self.__nom

    # Méthode get_age()
```

Questions

1. Pourquoi les attributs `__nom` et `__age` sont-ils privés ?
2. Comment accède-t-on à l'attribut `__nom` ?
3. Ajoutez une méthode `get_age` pour accéder à l'attribut `__age`.
4. Créez une instance de `Personne` de nom `"Luc"` et d'âge 25, puis affichez son nom.

Réponses

Exercice 7 — Ajouter un mutateur

```
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.__titulaire = titulaire
        self.__solde = solde

    def get_solde(self):
        return self.__solde
```

Questions

1. Que signifie l'encapsulation dans cette classe ?
2. Que fait la méthode `get_solde` ?
3. Créez un compte bancaire pour `"Emma"` avec un solde de 500 et affichez le solde.
4. Ajoutez une méthode `set_solde` pour modifier le solde.

Réponses

Exercice 8 — Encapsulation avec validation

```
class Etudiant:
    def __init__(self, nom, note):
        self.__nom = nom
        self.__note = note

    def get_note(self):
        return self.__note

    # Méthode set_note : notes entre 0 et 20
```

Questions

1. Pourquoi utiliser un mutateur pour la note ?
2. Que se passe-t-il si on essaie d'accéder directement à `__note` ?
3. Ajoutez une méthode `set_note` qui n'accepte que des notes entre 0 et 20.
4. Créez un étudiant nommé `"Paul"` avec une note de 15, puis essayez de changer la note à 25.

Réponses

Exercice 9 — Héritage simple

```python
class Animal:
    def __init__(self, nom):
        self.__nom = nom

    def get_nom(self):
        return self.__nom

    def faire_son(self):
        return "Son generique"

class Chat(Animal):
    # Méthode faire_son pour retourner "Miaou"
```

Questions

1. Quelle classe est la classe parent ?
2. Que fait l'héritage dans ce cas ?
3. Redéfinissez la méthode `faire_son` pour retourner `"Miaou"`.
4. Créez un `Chat` nommé `"Minou"` et affichez son son.

Réponses

Exercice 10 — Héritage avec nouvel attribut

```python
class Vehicule:
    def __init__(self, marque):
        self.__marque = marque

    def get_marque(self):
        return self.__marque

class Voiture(Vehicule):
    def __init__(self, marque, couleur):
        super().__init__(marque)
        self.__couleur = couleur

    # À compléter : méthode get_couleur
```

Questions

1. Que fait `super().__init__(marque)` ?
2. Combien d'attributs privés possède la classe `Voiture` ?
3. À compléter : ajoutez une méthode `get_couleur`.

Réponses

Exercice 11 — Encapsulation et mutateur avec validation

```python
class Livre:
    def __init__(self, titre, prix):
        self.__titre = titre
        self.__prix = prix

    def get_prix(self):
        return self.__prix

    # À compléter : set_prix refuse les prix négatifs
```

Questions

1. Pourquoi le prix est-il un attribut privé ?
2. Que fait un mutateur avec validation ?
3. Créez un `Livre` intitulé `"Python 101"` avec un prix de 30, puis essayez de définir un prix de -10.

Réponses